

## Principles of OOP

### Class

A **class** is a **blueprint or template** for creating objects. It defines the data (variables) and behaviour (methods) that objects will have.

### Object

An **object** is a real-world entity and an **instance of a class**. It occupies memory and can access the variables and methods defined in the class.

### Encapsulation (Data Hiding)

Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit (class) and restricting direct access to the data.

```
class Student {  
    private String name; // Private variable  
    // Setter method  
    public void setName(String name) {  
        this.name = name;  
    }  
    // Getter method  
    public String getName() {  
        return name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.setName("Alice");  
        System.out.println(s.getName());  
    }  
}
```

Alice

### Inheritance

Inheritance allows one class to acquire the properties and methods of another class using the extends keyword.

```
class Animal {
```

```

    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound(); // Inherited method
        d.bark();
    }
}

```

Animal makes a sound

Dog barks

## Polymorphism

Polymorphism means "**many forms.**" It allows the same method to perform different tasks.

There are two types:

- **Compile-time Polymorphism** (Method Overloading)
- **Run-time Polymorphism** (Method Overriding)

## Abstraction

Abstraction hides implementation details and shows only the essential features. In Java, abstraction is achieved using:

- **Abstract classes**
- **Interfaces**

```

abstract class Vehicle {
    abstract void start();
    void stop() {

```

```

        System.out.println("Vehicle stopped");
    }
}
class Car extends Vehicle {
    @Override
    void start() {
        System.out.println("Car started");
    }
}
public class Main {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v.stop();
    }
}

```

Car started

Vehicle stopped

```

interface Animal {
    void sound(); // Abstract method
}
class Dog implements Animal {
    @Override
    public void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}

```

Dog barks

| <b>Abstract Class</b>                  | <b>Interface</b>                                                                         |
|----------------------------------------|------------------------------------------------------------------------------------------|
| Uses extends                           | Uses implements                                                                          |
| Can have abstract and concrete methods | Traditionally contains abstract methods (Java 8+ also allows default and static methods) |
| Cannot create an object                | Cannot create an object                                                                  |
| Constructor is allowed                 | No constructor                                                                           |
| Can have instance variables            | Cannot have instance variables                                                           |